Java 17 is a **big one** — it's an **LTS (Long-Term Support)** release and widely used in real-world projects.

Below is a **clean + interview-ready breakdown** of **Java 17 features**, from **must-know** to **nice-to-know**.

✅ Java 17 (LTS) – Key Features

1 Sealed Classes (Finalized)

Control **which classes can extend or implement** a class/interface.

```
public sealed class Shape permits Circle, Rectangle { }

final class Circle extends Shape { }
final class Rectangle extends Shape { }
```

 Why?

- Better **security**
- More **predictable inheritance**
- Cleaner **domain modeling**

---

2 Pattern Matching for switch (Preview)

More powerful switch with **type checks + casting**.

```
static String test(Object obj) {
   return switch (obj) {
      case Integer i -> "Int: " + i;
      case String s  -> "String: " + s;
      default        -> "Unknown";
   };
}
```

 Why?

- Removes instanceof + cast
- Cleaner & safer code

---

3 Strong Encapsulation of JDK Internals

Internal APIs are **no longer accessible by default**.

IllegalAccessError at runtime

 Why?

- Improves **security**
- Prevents dependency on unsafe internals

✅ Use proper public APIs or --add-opens (only if unavoidable)

---

4 New macOS Rendering Pipeline

Java 2D uses **Apple Metal** instead of OpenGL.

 Why?

- Better performance on macOS
- Future-proof rendering

---

5 Foreign Function & Memory API (Incubator)

Interact with **native code** without JNI.

MemorySegment segment = MemorySegment.allocateNative(100);

 Why?

- Faster than JNI
- Safer memory access
- Useful for system-level programming

---

6 Deprecation for Removal

Several old APIs marked for removal:

- SecurityManager
- Applet API (already dead)
- RMI Activation

 Why?

- Cleaner JVM
- Modern security approaches

---

7 Enhanced Pseudo-Random Number Generators

New interfaces & implementations.

```
RandomGenerator rng = RandomGenerator.getDefault();
int n = rng.nextInt(100);
```

 Why?

- Better statistical quality
- Useful for simulations, ML, gaming

---

8 Context-Specific Deserialization Filters

More control over **deserialization attacks**.

```
ObjectInputFilter.Config.setSerialFilter(filter);
```

 Why?

- Improves security
- Prevents gadget attacks

---

9 Removal of Experimental AOT & JIT Compiler

Removed:

- Graal AOT (experimental)
- Legacy JIT options

 Why?

- Focus on stable JVM features

 Java 17 = Java 8 + Java 11 + Java 17

If you **skip-upgraded**, Java 17 includes:

- Lambda improvements
- Streams enhancements
- Records
- Text Blocks
- HTTP Client
- var
- Garbage Collector upgrades

□ Interview One-Line Summary

Java 17 is an LTS release focused on security, sealed classes, pattern matching, strong encapsulation, and modern JVM cleanup.

□ Want More?

I can also give:

- ✅ **Java 17 vs Java 8 vs Java 11 (table)**
- ✅ **Real-world use cases of Sealed Classes**
- ✅ **Java 17 interview questions**
- ✅ **Migration checklist (Java 8 → 17)**

Just tell me □

Sealed Classes shine when **you want strict control over inheritance** and **the domain has a fixed set of possibilities**.
Below are **real-world, production-grade use cases** you'll actually see in backend systems, APIs, and enterprise apps.

---

1 Payment Systems (Very Common)

□ Real-world scenario

A payment can only be **CreditCard**, **UPI**, or **NetBanking**.
No random developer should extend it with CryptoPayment □.

```
public sealed interface Payment
    permits CreditCardPayment, UPIPayment, NetBankingPayment { }

final class CreditCardPayment implements Payment { }
final class UPIPayment implements Payment { }
final class NetBankingPayment implements Payment { }
```

✅ **Benefits**

- Prevents unauthorized payment types
- Compile-time safety
- Cleaner switch logic

```
String process(Payment p) {
   return switch (p) {
      case CreditCardPayment c -> "Card Payment";
      case UPIPayment u       -> "UPI Payment";
      case NetBankingPayment n-> "Net Banking";
   };
}
```

 Used in: Banking apps, fintech platforms, payment gateways

---

2 Order Status in E-Commerce

 Real-world scenario

Order states are **finite and controlled**.

```
public sealed class OrderStatus
      permits Created, Shipped, Delivered, Cancelled { }

final class Created extends OrderStatus { }
final class Shipped extends OrderStatus { }
final class Delivered extends OrderStatus { }
final class Cancelled extends OrderStatus { }
```

✅ **Benefits**

- No invalid states like LostInSpace
- Safe state transitions
- Better business rule enforcement

 Used in: Amazon-like systems, logistics, supply-chain apps

---

3 API Response Modeling (REST / Microservices)

 Real-world scenario

An API response can only be **Success**, **ValidationError**, or **ServerError**.

```
public sealed interface ApiResponse
      permits Success, ValidationError, ServerError { }

record Success(String data) implements ApiResponse { }
record ValidationError(String message) implements ApiResponse { }
record ServerError(int code) implements ApiResponse { }
```

✅ **Benefits**

- Exhaustive handling
- No null responses
- Clean API contracts

 Used in: Spring Boot REST APIs, Microservices

---

4 Authentication & Authorization Systems

 Real-world scenario

Users have **strict roles**.

```
public sealed class UserRole
      permits Admin, Manager, Customer { }

final class Admin extends UserRole { }
final class Manager extends UserRole { }
final class Customer extends UserRole { }
void checkAccess(UserRole role) {
   switch (role) {
      case Admin a    -> allowAll();
      case Manager m -> allowLimited();
      case Customer c-> allowBasic();
   }
}
```

✅ **Benefits**

- Prevents role explosion
- Safer access control logic
- Cleaner RBAC implementation

 Used in: Enterprise security systems, IAM platforms

---

5 Notification Systems

 Real-world scenario

Notification channels are fixed.

```
public sealed interface Notification
    permits Email, SMS, Push { }

record Email(String address) implements Notification { }
record SMS(String number) implements Notification { }
record Push(String deviceId) implements Notification { }
```

✅ **Benefits**

- No unsupported notification types
- Easier to route messages
- Exhaustive handling

 Used in: Alerting systems, OTP services

---

6 Financial Instruments / Trading Platforms

 Real-world scenario

Financial instruments are limited.

```
public sealed class Instrument
    permits Stock, Bond, MutualFund { }

final class Stock extends Instrument { }
final class Bond extends Instrument { }
final class MutualFund extends Instrument { }
```

✅ **Benefits**

- Regulatory safety
- Predictable risk calculations
- Cleaner pricing engines

 Used in: Stock trading apps, risk engines

---

7 Compiler / Expression Trees (Advanced)

□ Real-world scenario

Expression types are known.

```
public sealed interface Expr
      permits Add, Subtract, Multiply { }

record Add(int a, int b) implements Expr { }
record Subtract(int a, int b) implements Expr { }
record Multiply(int a, int b) implements Expr { }
int eval(Expr e) {
   return switch (e) {
      case Add a     -> a.a() + a.b();
      case Subtract s-> s.a() - s.b();
      case Multiply m-> m.a() * m.b();
   };
}
```

□ **Used in:** Rule engines, DSLs, compilers

□ When NOT to Use Sealed Classes

❌ Plugin-based systems
❌ Framework extension points
❌ When inheritance must stay open
❌ Libraries meant for 3rd-party extensions

□ Interview Power Statement

Sealed classes are ideal when the domain has a closed set of types and business rules must be enforced at compile time.

□ Want Next?

I can give you:

- ✅ **Sealed vs Enum vs Interface (comparison table)**
- ✅ **Sealed + Records + Pattern Matching combo**
- ✅ **Java 17 interview questions on sealed classes**
- ✅ **Real Spring Boot project example**

Just say the word

Perfect timing — **Java 8 → Java 17 migration** is one of the **most asked real-world + interview topics** right now.

Below is a **battle-tested, step-by-step migration checklist** used in **enterprise production systems**.

✅ Java 8 → Java 17 Migration Checklist (Production-Ready)

---

1 Pre-Migration Assessment (MOST IMPORTANT)

Check Current State

✔Java version in prod

✔Build tool version (Maven / Gradle)

✔Framework versions (Spring, Hibernate, etc.)

✔Third-party dependencies

```
java -version
mvn -version
gradle -version
```

Rule:

If it runs on Java 8 but uses internal APIs → it will break in Java 17

---

2 Upgrade Build Tools FIRST

✅ Maven

Component	Minimum Version
Maven	3.8.1+
maven-compiler-plugin	3.10+

```
<properties>
  <maven.compiler.release>17</maven.compiler.release>
</properties>
```

✅ **Gradle**

Component	Minimum Version
Gradle	7.3+

```
java {
  toolchain {
    languageVersion = JavaLanguageVersion.of(17)
  }
}
```

---

3 Dependency Compatibility Check

□ **Most Common Breaking Libraries**

Library	Required Version
Spring Boot	2.7+ / 3.x
Hibernate	5.6+
Lombok	1.18.22+
Mockito	4+
JUnit	5
Jackson	2.13+

□ **Tip**

```
mvn dependency:tree
```

---

4 Remove Usage of JDK Internal APIs □

❌ **Java 8 code (Breaks in Java 17)**

```
sun.misc.Unsafe
com.sun.*
jdk.internal.*
```

✅ **Fix**

- Replace with standard APIs

- Temporary workaround (NOT recommended):

--add-opens java.base/java.lang=ALL-UNNAMED

 Rule:

Fix the code, don't rely on JVM flags in prod.

---

5 Handle Strong Encapsulation (Java 9+)

Common Runtime Errors

IllegalAccessError
InaccessibleObjectException

 Solution

- Upgrade libraries
- Remove reflective access
- Use supported APIs

---

6 Migrate Tests (JUnit 4 → JUnit 5)

✕ **Old**

@RunWith(SpringRunner.class)

✅ **New**

@ExtendWith(SpringExtension.class)

 Bonus: Parallel tests, better lifecycle hooks

---

7 Garbage Collector Review (Performance Boost)

Default GC

Java Version	GC
Java 8	Parallel / CMS

Java 17	G1 (default)

Optional

-XX:+UseZGC
-XX:+UseShenandoahGC

 Tip: Do performance testing before switching GC.

---

8 Enable & Adopt Modern Java Features (Optional but Recommended)

Replace Boilerplate

Java 8	Java 17
POJOs	Records
Enums + switch	Sealed + switch
String concat	Text Blocks

record User(int id, String name) {}

---

9 Security & TLS Changes

✔TLS 1.0 / 1.1 disabled

✔Stronger crypto defaults

 If legacy systems fail

-Djdk.tls.client.protocols=TLSv1.2

---

10 Logging & JVM Flags Cleanup

Remove Deprecated Flags

✕

-XX:+UseConcMarkSweepGC

✓

-XX:+UseG1GC

---

11 CI/CD Pipeline Update

✔Update Docker base image:

FROM eclipse-temurin:17-jdk

✔Update Jenkins / GitHub Actions Java version

---

12 Run Tools Before Production

 MUST-RUN Tools

jdeps --jdk-internals your-app.jar
mvn -U clean verify

 Detects forbidden APIs & dependency issues.

---

13 Staged Rollout Strategy (REAL WORLD)

✔Dev → QA → UAT → Prod

✔Canary release (5–10%)

✔Monitor:

- CPU
- Memory
- GC logs
- Latency

 Interview-Ready One-Line Answer

Java 8 to 17 migration requires build tool upgrades, dependency alignment, removal of internal API usage, handling strong encapsulation, and thorough testing before production rollout.

Want Add-Ons?

I can also give:

- ✅ **Java 8 vs 11 vs 17 migration path**
- ✅ **Common runtime errors & fixes**
- ✅ **Spring Boot Java 17 migration checklist**
- ✅ **Java 17 JVM flags for production**

Just tell me

Great choice — this combo is **pure Java-17 interview gold**
Let's break it into **two crisp sections**: **comparison table** first, then the **modern combo pattern**.

✅ Sealed vs Enum vs Interface (Comparison Table)

Aspect	Enum	Interface	Sealed Class / Interface
Purpose	Fixed constants	Contract / behavior	Controlled inheritance
Number of types	**Strictly fixed**	Unlimited	**Fixed but extensible**
Can hold state	❌ (limited)	❌	✅
Can have behavior	✅	✅ (default methods)	✅
Can restrict implementations	✅ (implicitly)	❌	✅ (permits)
Can be extended outside package	❌	✅	❌ (unless permitted)
Supports inheritance	❌	Multiple inheritance	Single inheritance
Best for	Status, constants	Framework APIs	Domain modeling
Exhaustive switch support	✅	❌	✅
Real-world flexibility	❌ Low	✅ High	✅ Balanced
Java version	Java 5	Java 8	Java 17

□ When to Use What (Interview Rule)

- **Enum** → *"Closed set of constants"*
- **Interface** → *"Open extension by third parties"*
- **Sealed** → *"Closed domain with behavior + safety"*

✅ Real-World Example Comparison

❌ Enum (Too rigid)

```
enum PaymentType {
   CARD, UPI, NET_BANKING
}
```

□ No behavior per type
□ Hard to evolve logic

❌ Interface (Too open)

```
interface Payment { }
```

□ Anyone can implement anything
□ No domain safety

✅ Sealed (Perfect balance)

```
public sealed interface Payment
      permits CardPayment, UPIPayment, NetBankingPayment { }
```

✔Controlled

✔Extensible

✔Safe

✅ Sealed + Records + Pattern Matching (□ Modern Java Combo)

This is **how Java 17 is meant to be written**.

□ Business Problem

A payment can be:

- Card
- UPI
- Net Banking

Each has **data + behavior**.

---

1 Sealed Interface (Domain Control)

```
public sealed interface Payment
    permits CardPayment, UPIPayment, NetBankingPayment { }
```

---

2 Records (Immutable Data Models)

```
record CardPayment(String cardNo, double amount) implements Payment {}

record UPIPayment(String upiId, double amount) implements Payment {}

record NetBankingPayment(String bank, double amount) implements Payment {}
```

✔No boilerplate

✔Immutable

✔Thread-safe

---

3 Pattern Matching + switch (Exhaustive & Safe)

```
static String process(Payment payment) {
  return switch (payment) {
    case CardPayment c ->
        "Card payment of " + c.amount();
    case UPIPayment u ->
        "UPI payment from " + u.upiId();
    case NetBankingPayment n ->
        "Net banking via " + n.bank();
  };
}
```

 Why this is powerful

- No default needed
- Compiler enforces completeness
- No casting
- No instanceof

 How Interviewers Love This Answer

Sealed classes define a closed domain, records model immutable data, and pattern matching ensures exhaustive and type-safe business logic handling.

✅ **Another Example: API Response Modeling**

```
public sealed interface ApiResponse
      permits Success, ValidationError, ServerError { }

record Success(String data) implements ApiResponse {}
record ValidationError(String message) implements ApiResponse {}
record ServerError(int code) implements ApiResponse {}
String handle(ApiResponse r) {
   return switch (r) {
      case Success s -> s.data();
      case ValidationError v -> v.message();
      case ServerError e -> "Error " + e.code();
   };
}
```

 Used heavily in:
Spring Boot APIs, Microservices, Clean Architecture

 When NOT to Use This Combo

❌ Plugin systems
❌ Framework extension APIs
❌ Libraries for unknown consumers

 Final One-Line Summary

Sealed + Records + Pattern Matching gives Java algebraic data types—clean, safe, and expressive domain modeling.

If you want next:

- ✅ **Spring Boot real project using this combo**
- ✅ **Java 17 interview Q&A**
- ✅ **Enum → Sealed migration examples**
- ✅ **Clean Architecture with sealed domains**

Just say □